

**МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ**  
**КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМ. ТАРАСА ШЕВЧЕНКА**  
**ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ**

**Кафедра мережевих та інтернет технологій**  
**БАЗА ДАНИХ ТА ІНФОРМАЦІЙНІ СИСТЕМИ**  
**РОЗШИРЕНІ МОЖЛИВОСТІ REDIS**

**Практична робота №9**

**Черкун Єви Сергіївни**

**Мета:** закріпити знання про роботу з Redis та ознайомитися з розширеним функціоналом — транзакціями, скриптами на Lua, публікацією/підпискою (Pub/Sub) та потоками Redis Streams

**Хід виконання роботи:**

**1. Транзакції в Redis**

**1.1. Проста транзакція додавання кількох ключів**

**Команди:**

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> SET key1 "value1"
QUEUED
127.0.0.1:6379(TX)> SET key2 "value2"
QUEUED
127.0.0.1:6379(TX)> INCR counter
QUEUED
127.0.0.1:6379(TX)> EXEC
```

**Результат:**

```
1) OK
2) OK
3) (integer) 1
127.0.0.1:6379> SET balance 100
OK
127.0.0.1:6379> WATCH balance
OK
127.0.0.1:6379> MULTI
OK
```

Успішно створено та виконано транзакцію, яка встановила значення двох ключів (key1, key2) та інкрементувала лічильник (counter).

## 1.2. Використання WATCH для імітації конкурентного доступу

Термінал 1:

```
127.0.0.1:6379> SET balance 100
OK
127.0.0.1:6379> WATCH balance
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> DECRBY balance 50
QUEUED
```

Термінал 2 (виконано між WATCH та EXEC у Терміналі 1):

```
127.0.0.1:6379> INCRBY balance 20
(integer) 120
```

Результат (Термінал 1):

```
127.0.0.1:6379(TX)> EXEC
(nil)
```

Продemonстровано використання команди WATCH. Перший термінал спостерігав за ключем balance. Другий термінал змінив значення цього ключа. В результаті, транзакція в першому терміналі була скасована (EXEC повернув nil), оскільки спостережуваний ключ було змінено

## 2. Lua-скрипти в Redis

### 2.1. Простий Lua-скрипт для перевірки та створення ключа

Команда:

```
127.0.0.1:6379> EVAL "if redis.call('exists', KEYS[1]) == 0 then return redis.call('set', KEYS[1], ARGV[1]) else return 'exists' end" 1 mykey "myvalue"
OK
```

Результат (якщо ключ testkey не існував):

OK

Результат (якщо ключ testkey вже існував):

"exists"

### 3. Механізм Pub/Sub

#### 3.1. Підписка на канал (Термінал 1)

Команда:

```
127.0.0.1:6379> SUBSCRIBE news
```

Результат:

Reading messages... (press Ctrl-C to quit)

- 1) "subscribe"
- 2) "news"
- 3) (integer) 1

#### 3.2. Публікація повідомлення (Термінал 2)

Команда:

```
127.0.0.1:6379> PUBLISH news "Redis is awesome!"
```

Результат:

(integer) 1

Результат (Термінал 1):

- 1) "message"
- 2) "news"
- 3) "Redis is awesome!"

### 4. Redis Streams (потoki даних)

#### 4.1. Додавання події до потоку

Команда (виконано в окремому терміналі, не в режимі підписки):

```
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 19.8
```

Результат:

"1746094335423-0"

До потоку mystream успішно додано нову подію з полями sensor-id та temperature.

## 4.2. Читання останніх подій

**Команда:**

```
127.0.0.1:6379> XRANGE mystream - +
```

**Результат (приклад):**

- 1) 1) "1682930542678-0"
- 2) 1) "sensor-id"
- 2) "1234"
- 3) "temperature"
- 4) "19.8"

## 4.3. Створення споживача потоку та зчитування нових повідомлень

**Команди:**

```
127.0.0.1:6379> XGROUP CREATE mystream mygroup 0 MKSTREAM  
OK  
127.0.0.1:6379> XREADGROUP GROUP mygroup consumer1 COUNT 2 STREAMS mystream >
```

**Результат (якщо є нові повідомлення):**

- 1) 1) "mystream"
- 2) 1) 1) "1682930542678-0"
- 2) 1) "sensor-id"
- 2) "1234"
- 3) "temperature"
- 4) "19.8"

**(Додатково) Скрипт (Python) для реалізації логіки Pub/Sub:**

```
import redis  
import time  
  
redis_host = 'localhost'  
redis_port = 6379  
redis_db = 0  
channel_name = 'news'  
  
try:  
    r = redis.Redis(host=redis_host, port=redis_port, db=redis_db,  
decode_responses=True)  
    pubsub = r.psubsub()  
    pubsub.subscribe(channel_name)
```

```

print(f"Підписано на канал '{channel_name}'. Очікування повідомлень...")

for message in pubsub.listen():
    if message['type'] == 'message':
        print(f"Отримано повідомлення: {message['data']}")
    elif message['type'] == 'subscribe':
        print(f"Успішно підписано на канал '{message['channel']}'")

except redis.exceptions.ConnectionError as e:
    print(f"Не вдалося підключитися до Redis: {e}")
except KeyboardInterrupt:
    print("Завершення скрипту.")
finally:
    if 'pubsub' in locals() and pubsub.subscribed:
        pubsub.unsubscribe(channel_name)
        print(f"Відписано від каналу '{channel_name}'.")

```

Після запуску скрипт підпишеться на канал news і чекатиме повідомлень.

Тепер відкрийте інший термінал redis-cli і опублікуйте повідомлення в канал news:

```

PS C:\Users\Ева\db\db-infosystems> python subscriber.py
Підписано на канал 'news'. Очікування повідомлень...

```

```

PS C:\Users\Ева\db\db-infosystems> docker exec -it redis-container redis-cli
127.0.0.1:6379> PUBLISH news "Redis is awesome from Docker!"
(integer) 1
127.0.0.1:6379>

```

```

Успішно підписано на канал 'news'
Отримано повідомлення: Redis is awesome from Docker!

```

## Висновок

У ході виконання практичних завдань було успішно ознайомлено з основними командами та принципами роботи транзакцій (MULTI, EXEC, DISCARD, WATCH), Lua-скриптів (EVAL), механізму Pub/Sub (SUBSCRIBE, PUBLISH) та Redis Streams (XADD, XRANGE, XREADGROUP). Було продемонстровано використання WATCH для імітації конкурентного доступу та базове використання кожного з нових механізмів Redis.